

YOLO26m C3k2/Bottleneck 硬體友善簡化

Codex 可執行研究計畫

Stage-Aware Lite-C3k2 + Basic W8A8 Quantization Robustness Check

版本 v3 | 2026-08-16 | 本文件用途：交由 Codex 直接閱讀、實作、訓練與彙整結果

本週只回答一個核心問題：在不改動 MASF、BinaryQK、PWL Softmax 的前提下，YOLO26m 的 C3k2/Bottleneck 可以簡化多少，且 FP32 精度與 W8A8 量化穩定性仍可接受？

0. Codex 執行原則

- 不要同時修改多個研究因素。C1、C2、C3 必須可單獨開關，才能判斷是哪一個修改造成精度變化。
- MASF/MFAM、BinaryQK、PWL Softmax 在本輪視為 frozen modules：不得重新設計、不得調參、不得與 C3k2 消融同時更動。
- 本輪不做 Concat→Add/Weighted Add，不做 INT4、不做 ternary、不加入 Dynamic/Deformable/SE/CBAM 等新模組。
- 所有候選使用相同 dataset split、imgsz、seed、augmentation、optimizer/scheduler 與 validation 流程。不得為個別 candidate 私下調 training recipe。
- 若目前 repo 的 class/path 與本文件名稱不同，先掃描現有實作並建立對照表；不要憑空建立第二套重複模組。
- 每個 checkpoint 訓練後必須 fresh-process reload 再驗證；不得只驗證記憶體中 monkey-patched model。

1. 研究範圍與成功條件

研究基準 C0 不是「乾淨官方 YOLO26m」重新開始，而是目前已整合完成且要保留的 YOLO26m 系統：

C0 = YOLO26m + frozen MASF/MFAM + frozen BinaryQK + frozen PWL Softmax
研究變因 = only C3k2 / C3k / Bottleneck structure
量化 = architecture freeze 後才做 Q0-Q2

主要評估指標依序為：mAP50-95、ball/small-object recall（若資料集提供）、GFLOPs/Params、實際 latency。AP_S 只有在 COCO 或具相容 small-object 標註時才使用，不要在自有資料集臆造。

精度門檻統一用「absolute percentage points (pp)」表示。Ultralytics 若輸出 0~1，小數差 0.008 就是 0.8 pp。

mAP50-95 下降	決策	Codex 行為
≤ 0.5 pp	PASS	優先保留；再比較 GFLOPs / latency。
0.5 ~ 0.8 pp	CONDITIONAL	只有在整體 GFLOPs 或實測 latency 改善約 ≥8% 時保留，否則淘汰。
> 0.8 pp	REJECT	原則上取消該修改，不進後續組合。若懷疑單次驗證噪聲，只標記為需複驗，不得直接宣稱失敗原因。

例：C0 mAP50-95 = 0.500，candidate = 0.492，差值 0.008 = 0.8 pp；若低於 0.492，即超過本輪預設淘汰線。這是本研究的工程門檻，不是宣稱為通用文獻標準。

2. 本週架構候選：只做 C0-C3，Rep 為條件式 R1

ID	唯一主要改動	第一輪放置位置	目的
C0	無 C3k2 簡化	現有架構	建立公平 reference。
C1	C3k2 hidden expansion e： 0.50 → 0.375	Backbone P4/P5 + Neck P4； P2/P3 保留	先做最安全的 width reduction。0.25 暫不列正式實驗。
C2	C3k inner Bottleneck depth： 2 → 1	同 C1；Neck P3 先保留	直接減少重複 spatial Conv。
C3	Bottleneck path：3x3 → 3x3 改為 1x1 → 3x3	採 mixed placement：P4/P5 + Neck P4；P3 保留原版	降低 spatial Conv 成本，同時保留一個 3x3。
R1	Rep recovery：在 C2 的 inner_n=1 基礎上，把剩餘 Bottleneck 換 RepBottleneck	只在 C2 有明顯計算優勢但掉 0.5~0.8 pp，或 GPU 時間允許 時執行	測試 Rep 是否能補回 C2 精度； 不是必要主線。

2.1 C1：只先測 e = 0.375

第一輪不要同時測 0.375 與 0.25。先確認 0.375 是否已能取得可觀成本下降且精度穩定。只有 C1 的 mAP50-95 下降 ≤ 0.5 pp，且仍有進一步壓縮需求時，才把 e=0.25 列為下一輪可選實驗，不占用本週核心時程。

2.2 C2：inner depth 2 → 1

保留 C3k2/CSP 外框、bypass、concat 與 residual，只將 C3k 內部重複 Bottleneck 從 2 個降為 1 個。這個實驗只回答「深度冗餘」問題，不同時更改 kernel 或 e。

2.3 C3：1x1 → 3x3，採 stage-aware mixed placement

C3 不做全網替換。主版本只改較深層 P4/P5 與 Neck P4，P2/P3/Neck P3 維持原 3x3→3x3，以降低 small-object feature 損失風險。

```
P2 / P3 / Neck P3 : keep 3x3 -> 3x3
P4 / P5 / Neck P4 : try 1x1 -> 3x3
Attention C3k2    : do not mix this kernel ablation with BinaryQK/PWL changes
```

若 C3 mixed 版本下降 > 0.8 pp，不要立刻做一堆組合；只允許一個 fallback：縮小替換範圍到 P5-only，再決定此方向是否值得保留。

2.4 R1：Rep-Bottleneck 的定位

R1 是 recovery candidate，不是新的主研究軸。Ultralytics 的 RepBottleneck 以 RepConv 取代 Bottleneck 的第一個 Conv；training 有多分支，部署前可 fuse。為了能和 C2 公平比較，R1 固定沿用 C2 的 inner_n=1，只改「standard Bottleneck vs RepBottleneck」。

R1 必須額外做 fuse 前後 numerical equivalence test；若 max absolute difference $> 1e-4$ ，先修正 fuse/reload 流程，不得進量化。由於 re-parameterization 可能增加 INT8 quantization sensitivity，只有 R1 在 FP32 有明顯價值時才做 Q0-Q2。

3. 訓練規則：100 epochs + early stopping，必要時再 +40

目標不是用短 epoch 排名，而是在一週內用一致規則把主要候選訓練到足夠收斂。3~5 epoch 只能做 smoke test。

階段	設定	決策規則
Smoke	3~5 epochs；只檢查 loss/gradient/shape/checkpoint/reload	禁止用 smoke mAP 排名。
正式 A	max_epochs=100；patience=20；相同 seed/retraining recipe	若 early stop，直接以 best.pt 評估，不再延長。
延長 B	最多再 +40 epochs；patience=15；從 last.pt resume，保留 optimizer/scheduler state	只有「未 early-stop 且尾段仍明顯改善」才進 B。

「尾段仍明顯改善」請用可重現規則判斷：滿足任一條件即可延長 40 epochs：

- best epoch 落在第 85~100 epoch；或
- 第 81~100 epoch 的 rolling-best mAP50-95 比第 61~80 epoch 至少再提高 0.1 pp。

若兩條都不滿足，就不延長。若進入 +40 後 early stop 觸發，立即停止，不必硬跑滿 140。

3.1 固定 training recipe

- 以目前 repo 可用的 YOLO26m pretrained checkpoint 初始化；能 shape-match 的權重必須轉移，missing/unexpected keys 要輸出成 JSON/文字報告。
- imgsz 預設 640；dataset split 不變。若自有資料來自影片，必須依 video/session/camera 分組，避免相鄰幀洩漏。
- 沿用目前 YOLO26 訓練 recipe / resolved args，不為單一 candidate 調 optimizer、LR、augmentation。Codex 必須保存每次 run 的完整 resolved args。
- 本週 architecture screening 使用 1 個固定 seed。3-seed 重複驗證留給最終 finalists，不要在一週內把 GPU 預算花在所有候選的多 seed。

4. 模型選擇與是否組合

本週完成 C0、C1、C2、C3 後，先選「單因子 Pareto 最佳」；不要預設一定要把所有方法合併。

情況	下一步
C1/C2/C3 只有一個 PASS	直接把它標成 C_best，進 Q0-Q2。
有兩個以上 PASS	比較 mAP drop、GFLOPs、latency；先選一個 C_best。組合實驗可留下一週。
C2 在 0.5~0.8 pp，但成本改善明顯	可觸發 R1，判斷 Rep 能否追回精度。
全部 >0.8 pp	不要硬組合；回到 C0，檢查放置策略或只保留最保守 C1。

本週「成功」不要求 MASF + Lite-C3k2 + Rep + 極低位元全部同時成立。只要找到至少一個 C3k2 簡化候選在 ≤ 0.8 pp 精度損失下帶來有意義的成本下降，並完成 W8A8 robustness check，就完成本輪研究目的。

5. 量化究只做 Q0-Q2：確認 Quantization Robustness

這一輪不要研究「哪種量化最好」，只回答：「C_best 相比 C0 是否因架構簡化而更怕 INT8？」

ID	內容	要回答的問題
Q0	Fused FP32 reference	部署 graph 的 FP32 精度是多少？
Q1	W8A8 PTQ：calibration 後直接 INT8 / bit-true 模擬，不重新訓練	模型天然對 INT8 有多敏感？
Q2	W8A8 QAT：插入 fake quant 後短期 fine-tune，再 convert/evaluate	量化誤差是否可以用基本 QAT 救回？

5.1 QAT 用最基本的方式可

QAT (Quantization-Aware Training) 不代表訓練時真的全部用 INT8 算。它是在 forward 中插入 fake quantization：把 float weight/activation 依 INT8 scale 做 round、clip，再 dequantize 回 float，讓模型在訓練時看見量化誤差；backward 仍可用梯度更新。PyTorch/TorchAO 的正式 QAT 流程也採 prepare(fake quant) → train/fine-tune → convert。

```
FP32 best.pt
-> prepare QAT / insert fake quantizers
-> fine-tune with simulated W8A8 numerics
-> freeze observers after ranges stabilize
-> convert to quantized model (or bit-true fake-quant eval if backend cannot convert
custom ops)
-> evaluate with the same validation set
```

Q2 建議只跑 10~15 epochs、patience=5，learning rate 使用 architecture fine-tuning LR 的約 0.1 倍。前 3 epochs 允許 observers 更新 activation range，之後 freeze observer；若框架已有可靠預設則優先使用框架流程，不要自己重寫整套量化器。

5.2 Q1/Q2 的量化範圍

- 目標是 W8A8：regular Conv/Linear weights 與支援的 activations 使用 INT8。不要在本輪再測 INT4/ternary。
- BinaryQK 已有自己的 binary arithmetic，不在 Q1/Q2 再二次量化；PWL Softmax 維持既有版本。
- MASF 中若是標準 Conv 且量化 backend 支援，可跟其他 Conv 使用同一 W8A8 規則；若 custom op 無法 convert，必須記錄 quantized/unquantized node list。
- 如果 backend 不能把整個 YOLO custom graph 真正 convert 成 INT8，可使用 fake-quant bit-true evaluation 做 robustness comparison，但報告必須寫 simulation_only=true，不能宣稱已完成實際 INT8 deployment。

5.3 Quantization robustness 判讀

指標	定義（純文字）	判讀
PTQ gap	Q0 mAP50-95 - Q1 mAP50-95	越小代表不經 retraining 也耐量化。
QAT gap	Q0 mAP50-95 - Q2 mAP50-95	本輪主要 robustness 指標。

QAT recovery	Q2 mAP50-95 - Q1 mAP50-95	看 QAT 能救回多少 PTQ 損失。
--------------	---------------------------	---------------------

QAT gap 建議： ≤ 0.5 pp = robust； $0.5 \sim 0.8$ pp = acceptable but sensitive； > 0.8 pp = quantization-sensitive。
Q1 PTQ 即使掉得較多，只要 Q2 能回到 ≤ 0.8 pp，仍可標記為「PTQ-sensitive but QAT-recoverable」。

6. Codex 必須完成的工程項目

項目	最低要求
Custom block API	LiteC3k2/LiteC3k 必須可明確設定 e、inner_n、kernel_mode、use_rep；不要依 parser 隱式覆寫。
Graph assertion	build 後列印 target layers 的 class、channels、kernel、inner_n；assert C1/C2/C3 真正生效。
Shape test	640x640 dummy input forward 可通過，Detect strides/outputs 正確。
Checkpoint test	save → 關閉 process → reload → val；custom module 仍存在。
Rep fuse test	R1 如執行：eval 模式下 fuse 前後 max_abs_diff $\leq 1e-4$ 。
Profiling	每個候選輸出 Params、GFLOPs/MAC（至少一種一致方法）、latency；若可行再加 peak activation memory。
Result manifest	保存 git commit、config hash、seed、resolved train args、best epoch、stop reason、metrics。

建議輸出結構（若 repo 已有既定 convention，沿用現有 convention，不必硬改）：

```
research_lite_c3k2/
  configs/      c0.yaml c1_e0375.yaml c2_n1.yaml c3_mixed_1x1_3x3.yaml [r1_rep.yaml]
  tests/        test_graph.py test_reload.py test_rep_fuse.py
  runs/<ID>/     args.yaml metrics.json profile.json best.pt last.pt
  quant/<ID>/    q0_fp32.json q1_ptq_w8a8.json q2_qat_w8a8.json quant_scope.json
  results.csv
  REPORT.md
```

7. 一週執行順序

日程	工作	必須產出
Day 1	掃描 repo、固定 C0、建立 LiteC3k2 API、graph/reload unit tests	C0 可重載；C1-C3 graph assertion 通過。
Day 2	C0/C1/C2/C3 smoke；修正 shape/transfer 問題後啟動正式訓練	smoke log + transfer report。
Day 3-4	完成主要 100-epoch/early-stop runs；依統一規則決定是否 +40	best/last + stop reason + metrics。
Day 5	彙整 C0-C3；必要時跑 C3 P5-only fallback 或條件式 R1（二選一，勿都擴張）	C_best 決策表。
Day 6	只對 C0 與 C_best 做 Q0/Q1/Q2	PTQ gap、QAT gap、recovery。

Day 7	重跑 fresh-process val/profile，整理 REPORT.md 與 results.csv	可直接放週報/論文的結果表。
-------	---	----------------

若只有單 GPU 且 100-epoch runs 無法在一週完成，優先順序固定為 C0 → C1 → C2 → C3 → R1。R1 永遠可以延後，不得為了跑 Rep 犧牲 C0-C3 的公平正式訓練。

8. 最終 REPORT.md 必須回答的問題

- C1 e=0.375 是否能在 ≤ 0.8 pp 損失下帶來值得的成本下降？是否有必要下探 0.25？
- C2 inner_n=1 與 C3 mixed 1x1→3x3，哪一種 redundancy reduction 的 accuracy/cost trade-off 更好？
- C3 的 stage-aware placement 是否比全網替換更適合小目標？若 mixed 失敗，P5-only fallback 是否改善？
- 若 R1 執行：Rep 是否能相對 C2 回復 FP32 精度？fuse 後是否保持數值一致？
- C_best 的 QAT gap 是否比 C0 顯著惡化？若有，是 PTQ-sensitive 還是 QAT 也無法追回？
- 本週下一步只能從結果推導：e=0.25、候選組合、INT4/ternary 都屬下一階段，不能在沒有 C0-C3 結果前先宣稱。

9. Codex 需要參考的來源

- [1] Ultralytics YOLO26 official YAML (current main) <https://github.com/ultralytics/ultralytics/blob/main/ultralytics/cfg/models/26/yolo26.yaml>
- [2] Ultralytics YOLO architecture guide: C2f/C3k2/C3k/Bottleneck <https://github.com/ultralytics/ultralytics/blob/main/docs/en/guides/yolo-architecture.md>
- [3] Ultralytics block.py: Bottleneck / C3k2 / RepBottleneck <https://github.com/ultralytics/ultralytics/blob/main/ultralytics/nn/modules/block.py>
- [4] YOLO26 paper: Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models <https://arxiv.org/abs/2606.03748>
- [5] QARepVGG: quantization risk of structural re-parameterization <https://arxiv.org/abs/2212.01593>
- [6] PyTorch/TorchAO QAT workflow: prepare fake quant → train → convert <https://docs.pytorch.org/ao/stable/workflows/qat.html>

最後提醒：本文件的 0.5/0.8 pp、8% 成本改善、QAT 10~15 epochs 等數值是本研究為了在一週內做出可比較結果所設定的工程門檻，不應在論文中描述成普遍公認標準。正式論文需以實驗結果與多 seed variance 再修訂。